AI CYBER THREAT DETECTION PLATFORM

Comprehensive Operations Manual, Architecture Specification & Viva Defense Dossier

Project Level: Final Year Major Project (7th/8th Sem)	**Classification Benchmark:** CIC-IDS2017 Flow Telemetry
Core Stack: Python, FastAPI, React 19, Scikit-Learn, SHAP, WebSockets	**Key Defense Features:** PCAP Parsing, Explainable AI, Active Netsh SOAR

1. System Architecture & End-to-End Workflow

The platform operates as a modern **Security Operations Center (SOC)** with an automated **Intrusion Detection and Prevention (IDS/IPS)** pipeline. The workflow bridges raw network packet capture, conversational flow reconstruction, machine learning inference, explainable feature attribution, generative AI triage, and active firewall containment.

Subsystem / Tier	Underlying Technology	Core Responsibility in Project

Ingestion Layer	`pcap_analyzer.py`
	`live_sniffer.py`
	`POST /pcap/upload`
		Extracts raw packet bytes via struct.unpack, parses Ethernet/IP/TCP headers, groups conversations into 5-tuple bidirectional flows, and computes 22 statistical flow features matching CICFlowMeter standards.

Machine Learning Tier	`RandomForestClassifier`
	`HistGradientBoosting`
	`Multi-Layer Perceptron (MLP)`
	`StandardScaler`
		Multi-model classifier trained on 12,000 CIC-IDS2017 records. Random Forest achieves 99.50% test accuracy and 99.37% Macro F1 with 16.9 us single-flow inference latency across 6 distinct traffic classes.

Explainable AI (XAI)	`shap.TreeExplainer` `backend/app/xai_service.py`	Computes local Shapley additive attributions for each alert, mathematically quantifying which exact flow metrics (SYN count, duration, packet length variance) triggered the alarm.
Generative AI Tier	`Google Gemini 1.5 Flash` `backend/app/ai_service.py`	Acts as an automated Tier-1 SOC analyst, translating technical flow metrics and MITRE technique codes into plain-English incident summaries and actionable containment playbooks.

Serving & Streaming	`FastAPI REST Endpoints`
	`WebSocket (/ws/threats)`
	`SQLAlchemy ORM`
		Provides sub-millisecond REST JSON endpoints and bidirectional WebSocket streams that push live alerts and PCAP ingestion results to connected analyst dashboards with zero latency.

Active Containment (SOAR)	`Windows Netsh AdvFirewall` `IPTables & Suricata Exporter`	Enforces host OS firewall packet drop rules on adversary IPs and generates ready-to-deploy Linux IPTables and Suricata IDS signature syntax directly from the investigation drawer.
Reporting & Forensics	`ReportLab Engine` `backend/app/report_service.py`	Generates 1-click formal executive incident forensic reports in PDF format with incident telemetry, severity rating, MITRE tags, and digital analyst sign-off.

2. Step-by-Step Operating & Demonstration Manual

Follow these exact steps during your evaluation to execute a flawless live presentation for the external examiners:

Step 1: Start Backend API & WebSockets

```
python -m uvicorn backend.app.main:app --port 8000 --reload
```

Starts the FastAPI service on port 8000 with real-time WebSocket connection manager, REST endpoints, and dynamic ML model pipeline.

Step 2: Start React Frontend Client

```
cd frontend && npm run dev
```

Launches the Vite React dashboard on http://localhost:5173/. Verify that the green 'WS STREAM SYNC' badge lights up in the top header.

Step 3: Web UI Drag-and-Drop PCAP Analysis (The Showstopper)

```
Navigate to Alerts -> Click 'Ingest PCAP Capture' -> Click 'Analyze Demonstration PCAP (1-Click)'
```

Demonstrates live binary packet capture ingestion. The pure-Python engine parses 15 raw frames via struct.unpack, reconstructs bidirectional conversations, runs ML inference, and displays classified threat flows with MITRE technique codes.

Step 4: Explainable AI (SHAP) & Forensic PDF Export

```
Click any threat alert -> Click 'Calculate SHAP' -> Click 'PDF Report'
```

In the investigation drawer, the SHAP horizontal bar chart visually breaks down feature weights (red = attack, green = benign). Clicking 'PDF Report' immediately downloads the formal multi-page forensic audit report.

Step 5: Multi-Model Benchmark Verification

```
Click 'Models (AI)' in the sidebar
```

Presents the comparative benchmark table showing Random Forest (99.50%), Hist Gradient Boosting, MLP Neural Network, and Logistic Regression with latency in microseconds.

Step 6: Live Hardware Sniffing Demonstration

```
python src/live_sniffer.py --count 10
```

Sniffs active wire packets directly off the host network card and streams live alerts into the dashboard header via WebSockets with zero page refresh.

3. Computer Networks (CN) Deep-Dive & Viva Q&A

Examiners frequently probe low-level networking concepts to verify that you understand what happens on the physical wire:

Q1: What is a Network Flow, and what is the 5-Tuple?

A network flow is an unidirectional or bidirectional sequence of packets transferred between two endpoints within a given time window sharing the same 5-tuple: (1) Source IP Address, (2) Destination IP Address, (3) Source Port Number, (4) Destination Port Number, and (5) Transport Layer Protocol (e.g., TCP or UDP). In our system, the PCAP parser groups individual packets by this 5-tuple to calculate conversation-level statistical features matching the CICFlowMeter standard.

Q2: How does the TCP Three-Way Handshake work, and how does a SYN Flood attack abuse it?

In a normal TCP connection establishment: (1) Client sends SYN (Synchronize) packet with initial sequence number x. (2) Server responds with SYN-ACK (Synchronize-Acknowledge) with seq y and ack x+1, allocating memory in its 'half-open connection backlog queue'. (3) Client responds with ACK (Acknowledge) with ack y+1.

In a SYN Flood (MITRE T1498), the attacker sends thousands of spoofed SYN packets but never returns the final ACK. The server's backlog queue becomes exhausted waiting for timeouts, causing denial of service for legitimate users. Our model detects this by monitoring 'syn_flag_cnt', 'down_up_ratio' (~0.0), and flow inter-arrival times.

Q3: Explain the structure of an IPv4 and TCP Header and how your PCAP parser extracts them.

An Ethernet frame has a 14-byte header: Dest MAC (6B), Src MAC (6B), EtherType (2B). If EtherType == 0x0800, the payload is IPv4.

The IPv4 header is minimum 20 bytes: Byte 0 (Version + IHL), Byte 9 (Protocol: 6 for TCP, 17 for UDP), Bytes 12-15 (Source IP), Bytes 16-19 (Destination IP).

The TCP header is minimum 20 bytes: Bytes 0-1 (Source Port), Bytes 2-3 (Destination Port), Bytes 4-7 (Sequence Number), Byte 13 (Control Flags: URG, ACK, PSH, RST, SYN, FIN).

In `src/pcap_analyzer.py`, we use Python's `struct.unpack('!BBHHHBBH4s4s', ip_data)` with network byte order ('!' = big-endian) to parse header fields without relying on external libraries.

Q4: What are the 6 standard TCP Control Flags and their operational purposes?

(1) SYN (Synchronize): Initiates connection establishment.
(2) ACK (Acknowledge): Confirms receipt of transmitted data.
(3) RST (Reset): Abruptly terminates a connection due to an unrecoverable error or rejected port connection (frequently observed during PortScans).
(4) FIN (Finish): Gracefully closes the connection in one direction.
(5) PSH (Push): Requests receiver to push buffered data immediately to the application layer.
(6) URG (Urgent): Indicates that payload contains urgent high-priority data.

Q5: What is the difference between Stateless and Stateful Firewall Packet Inspection?

A Stateless Firewall examines individual packets in isolation based purely on static rules (e.g., block all traffic from 198.51.100.10). It has no concept of connection state.

A Stateful Firewall tracks the state of active network connections (NEW, ESTABLISHED, RELATED, INVALID) using a connection tracking table. It automatically permits inbound response packets if they belong to an established outbound connection. When our platform blocks an IP via Windows `netsh advfirewall`, it injects a stateful drop rule into the host filtering engine.

Q6: What is a PCAP file and what is its internal binary structure?

A PCAP (Packet Capture) file stores raw network packet data captured by libpcap/WinPcap/Npcap. It begins with a 24-byte Global Header containing a 4-byte Magic Number (0xa1b2c3d4 for standard microsecond timestamps, 0xa1b23c4d for nanosecond timestamps), major/minor version (2.4), snapshot length, and data link type (1 = Ethernet). Each packet has a 16-byte Packet Header containing timestamp seconds, microseconds, captured length, and original length, followed immediately by the raw packet frame bytes.

Q7: What is Promiscuous Mode and Raw Sockets in socket programming?

Normally, a Network Interface Card (NIC) discards all packets whose destination MAC address does not match its own MAC address or broadcast. In Promiscuous Mode, the NIC passes all packets traversing the local network segment directly to the OS kernel. A Raw Socket (SOCK_RAW) bypasses transport-layer TCP/UDP parsing and allows an application to read and construct custom IP and Ethernet headers directly. In `src/live_sniffer.py`, we initialize `socket(AF_INET, SOCK_RAW, IPPROTO_IP)` and invoke `ioctl(SIO_RCVALL, RCVALL_ON)` on Windows.

4. Machine Learning (ML) Foundations & Viva Q&A

Be prepared to answer theoretical questions on model architectures, mathematical formulas, and evaluation tradeoffs:

Q1: How does a Random Forest Classifier work mathematically and algorithmically?

Random Forest is an ensemble meta-estimator that fits multiple decision tree classifiers on various sub-samples of the dataset and uses majority voting to control overfitting. It combines two core techniques:

(1) Bagging (Bootstrap Aggregating): Each tree is trained on an independent bootstrap sample of size N drawn with replacement from the training set.
(2) Random Subspace Method: At each node split, only a random subset of features (typically sqrt(p), where p=22 in our project) is evaluated.

The final prediction for an unseen sample x is given by the mode of individual tree predictions: y_pred = mode{T_1(x), T_2(x), ..., T_B(x)}.

Q2: What is Gini Impurity, and how is it calculated?

Gini Impurity measures the frequency with which a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset. For a node with C classes, the formula is:

Gini = 1 - sum(p_i^2) for i=1 to C

where p_i is the proportion of samples belonging to class i. A pure node containing only one class has Gini = 0. The tree split algorithm chooses the feature and split threshold that maximizes the Reduction in Impurity (Information Gain): Delta_Gini = Gini_parent - [(N_left/N)*Gini_left + (N_right/N)*Gini_right].

Q3: Why did you use StandardScaler instead of MinMaxScaler?

MinMaxScaler squashes feature values into [0, 1] based strictly on min and max observed values: x_scaled = (x - min) / (max - min). If a dataset contains volumetric network anomalies (e.g. DDoS flows with millions of bytes/sec), an extreme outlier will compress 99% of normal traffic into a tiny range near 0.

StandardScaler computes: z = (x - mu) / sigma, centering the mean to 0 with unit variance. It preserves the relative distance of extreme outliers and prevents numerical instability during gradient calculations.

Q4: Explain Precision, Recall, Macro F1, and Micro F1. Why is Macro F1 critical in cybersecurity?

Precision = TP / (TP + FP) measures the correctness of positive alarms (minimizing false alarms). Recall = TP / (TP + FN) measures the proportion of actual attacks detected (minimizing false negatives). F1-Score = 2 * (Precision * Recall) / (Precision + Recall) is the harmonic mean.

In unbalanced datasets, Micro F1 pools all TP, FP, FN across all classes, which gets dominated by the majority class (Benign traffic). Macro F1 calculates the F1-score independently for each class and takes an unweighted average. A high Macro F1 (99.37% in our project) guarantees that rare, high-severity attacks like Web Attacks and Botnets are detected just as accurately as high-volume DDoS traffic.

Q5: How did you handle Class Imbalance during training?

Real-world network traffic is naturally imbalanced (benign traffic significantly outnumbers zero-day exploits). We handled this through three complementary methods:

(1) Stratified Splitting: `train_test_split(..., stratify=y)` ensures that the exact class proportions are maintained identically in both train (9,600) and test (2,400) sets.
(2) Cost-Sensitive Class Weighting: `RandomForestClassifier(class_weight='balanced')` computes weights inversely proportional to class frequencies: w_j = N / (C * n_j), penalizing misclassifications of minority attack classes heavily.
(3) Macro-Averaged Evaluation: Tracking Macro F1 ensures minority classes are not masked.

Q6: What is Data Leakage in ML, and how did you prevent it in your data pipeline?

Data leakage occurs when information from outside the training dataset (specifically test set target labels or future data) inadvertently influences model creation. In network intrusion detection, common leakage vectors include:

(1) Fitting StandardScaler on the entire dataset before splitting (we fit strictly on X_train and only transform X_test via Pipeline).
(2) Including identifiers like timestamp, flow ID, or specific internal IP addresses as input features (our model trains strictly on the 22 mathematical flow metrics, ensuring generalization across arbitrary network topologies).

Q7: Explain the difference between Bagging (Random Forest) and Boosting (HistGradientBoosting).

In Bagging (Random Forest), trees are grown independently in parallel on bootstrap subsets; each tree is deep and has high variance but low bias, and ensemble averaging reduces total variance without increasing bias.

In Boosting (Gradient Boosting), trees are trained sequentially in series; each new tree is shallow (high bias, low variance) and is fitted directly on the pseudo-residuals (negative gradients of the loss function) of the previous trees, iteratively reducing bias.

5. Deep Learning (DL) Foundations & Viva Q&A

Examiners frequently test whether you understand the internal mechanics of Neural Networks and why you selected your model architecture:

Q1: What is a Multi-Layer Perceptron (MLP), and how does Forward Propagation work?

An MLP is a feedforward artificial neural network consisting of an input layer, one or more hidden layers, and an output layer. In our comparative benchmark, the MLP has hidden layers of sizes (64, 32). In forward propagation, each neuron computes a linear combination of its inputs followed by a non-linear activation: z^[l] = W^[l] * a^[l-1] + b^[l], and a^[l] = g(z^[l]), where W is the weight matrix, b is the bias vector, and g is the activation function (e.g. ReLU).

Q2: Explain Backpropagation and the Chain Rule mathematically.

Backpropagation computes the gradient of the loss function L with respect to every weight in the network via the calculus Chain Rule. For a weight W_ij connecting neuron j in layer l-1 to neuron i in layer l:

dL/dW_ij = (dL/dz_i^[l]) * (dz_i^[l]/dW_ij) = delta_i^[l] * a_j^[l-1]

where delta_i^[l] is the error term. Gradients propagate backwards from the output layer to the input layer, and weights are updated using gradient descent: W = W - eta * (dL/dW), where eta is the learning rate.

Q3: What Activation Functions did you use, and why not use Sigmoid in hidden layers?

In hidden layers, we use ReLU (Rectified Linear Unit): f(x) = max(0, x). Sigmoid: f(x) = 1 / (1 + e^-x) suffers from the Vanishing Gradient Problem because its derivative f'(x) = f(x)*(1 - f(x)) maxes out at 0.25 and approaches 0 for |x| > 4. Multiplying many small fractions across layers causes gradients to shrink exponentially, stalling early layer learning.

In the output layer for multi-class classification, we use Softmax: P(y=c|x) = e^(z_c) / sum(e^(z_k)), converting raw logits into a valid probability distribution summing to 1.0.

Q4: What is the Loss Function for Multi-Class Classification?

Categorical Cross-Entropy Loss: L = - sum_{c=1}^C y_c * log(p_c), where y_c is the binary ground-truth indicator (1 for true class, 0 otherwise) and p_c is the model's Softmax probability for class c. It heavily penalizes confident incorrect predictions with logarithmic loss (e.g., if true class probability is 0.01, loss is -log(0.01) = 4.6).

Q5: How does the Adam Optimizer work compared to standard Stochastic Gradient Descent (SGD)?

Standard SGD updates weights with a fixed learning rate: W = W - eta * g_t. Adam (Adaptive Moment Estimation) computes individual adaptive learning rates for each parameter by calculating exponentially decaying moving averages of past gradients (first moment / momentum m_t) and past squared gradients (second moment / uncentered variance v_t):

m_t = beta_1 * m_{t-1} + (1 - beta_1)*g_t
v_t = beta_2 * v_{t-1} + (1 - beta_2)*g_t^2

After bias correction, the weight update is: W = W - [eta / (sqrt(v_hat_t) + eps)] * m_hat_t, providing fast convergence across disparate feature scales.

Q6: Why did Random Forest outperform Deep Learning (MLP) on your Network Flow dataset?

This is a fundamental machine learning reality documented in academic research (e.g., Grinsztajn et al., NeurIPS 2022: 'Why do tree-based models still outperform deep learning on tabular data?'):

(1) Tabular Inductive Bias: Network telemetry consists of structured tabular columns with heterogeneous distributions and sharp boundary thresholds (e.g., 'syn_flag_cnt == 1' and 'tot_bwd_pkts == 0'). Decision tree splits are axis-aligned step functions that partition this feature space naturally, whereas neural networks struggle to represent sharp coordinate cuts without smoothing.

(2) Latency & Efficiency: Random Forest runs in 16.9 us with instant 0.63s training, while the MLP required 9.95s training and achieved slightly lower Macro F1 (98.73% vs 99.37%).

Q7: How could an Autoencoder or 1D-CNN be used for Zero-Day threat detection?

An Autoencoder is an unsupervised neural network trained strictly on normal (Benign) traffic to compress flows into a low-dimensional bottleneck (encoder) and reconstruct them (decoder). When an unseen zero-day exploit arrives with abnormal telemetry, the network fails to reconstruct it, resulting in a high Reconstruction Error (Mean Squared Error). If MSE exceeds a calibrated threshold, the packet is flagged as an anomaly. A 1D-CNN can scan sequential packet payload bytes using 1D convolutional kernels to detect signature byte sequences regardless of packet alignment.

6. Explainable AI (XAI) & Generative AI Viva Q&A

Examiners love asking about interpretability because black-box models are unacceptable in regulated security operations:

Q1: What is the Black-Box problem in Security Operations Centers, and why is Explainable AI (XAI) needed?

In high-stakes security operations, an alert stating 'Classification: DDoS (Confidence 99%)' without context creates alert fatigue and distrust. Tier-1 analysts cannot verify whether the classifier detected a genuine attack or flagged a false positive caused by benign server backups. Explainable AI bridges this trust deficit by mathematically showing the exact features and thresholds responsible for the alert, allowing rapid verification and compliant forensic reporting.

Q2: What is SHAP, and how are Shapley Values calculated mathematically?

SHAP (SHapley Additive exPlanations) is rooted in cooperative game theory (Lloyd Shapley, 1953). It treats feature values as 'players' in a game where the prediction is the 'payout'. The Shapley value phi_i allocates payout to feature i by computing its marginal contribution across all possible feature subsets S subset of F \ {i}:

phi_i = sum_{S subset of F \ {i}} [|S|!(|F| - |S| - 1)! / |F|!] * [f(S union {i}) - f(S)]

SHAP is uniquely guaranteed to satisfy three foundational properties: Local Accuracy (sum of feature attributions equals prediction difference from base rate), Missingness (missing features have zero attribution), and Consistency (if a model changes such that a feature contributes more, its attribution does not decrease).

Q3: Why did you use TreeExplainer instead of KernelExplainer or LIME?

KernelExplainer and LIME (Local Interpretable Model-agnostic Explanations) are model-agnostic methods that rely on sampling perturbations, which makes them slow (several seconds per prediction) and non-deterministic. TreeExplainer (Lundberg et al., Nature Machine Intelligence 2020) is an exact polynomial-time algorithm tailored for decision trees that traverses tree pathways in O(T * L * D^2) time (where T=trees, L=leaves, D=depth). In our platform, TreeExplainer computes exact feature attributions in under 15 milliseconds, enabling real-time interactive rendering in the web browser.

Q4: What is the role of Google Gemini 1.5 Flash in your platform, and how does it prevent hallucinations?

Google Gemini serves as an automated Tier-1 incident response advisor. To prevent LLM hallucinations, we employ strict Grounded Prompt Engineering: the prompt includes the exact observed numerical flow telemetry, the MITRE ATT&CK technique code, and system asset tags. Gemini is instructed to synthesize a concise, deterministic two-sentence briefing: Sentence 1 explains the root-cause anomaly in plain English, and Sentence 2 provides an actionable SOC triage playbook (e.g. 'Isolate IP 185.220.101.42 and verify web server auth logs'). If the API is unreachable or times out, the backend automatically falls back to pre-compiled local heuristic security playbooks.

7. Cybersecurity, SOAR & Defense-in-Depth Viva Q&A

Examiners frequently evaluate your practical understanding of Security Orchestration, Automation, and Response:

Q1: What is the difference between SIEM, SOAR, IDS, and IPS?

(1) IDS (Intrusion Detection System): Passively monitors traffic and triggers alerts when threats are identified (passive).
(2) IPS (Intrusion Prevention System): In-line network appliance that actively drops packets or terminates connections when attacks occur (active).
(3) SIEM (Security Information and Event Management): Centralized data aggregator collecting, correlating, and analyzing log telemetry across all network endpoints and firewalls.
(4) SOAR (Security Orchestration, Automation, and Response): Automates response playbooks (e.g. automatically isolating a compromised host or blocking a firewall IP) to reduce Mean Time to Respond (MTTR). Our platform combines IDS analytics with automated SOAR containment.

Q2: What is the MITRE ATT&CK; Framework, and how did you map it in your project?

MITRE ATT&CK; (Adversarial Tactics, Techniques, and Common Knowledge) is a globally accessible knowledge base of adversary behaviors based on real-world observations. It breaks attacks down into Tactics (the adversary's technical goal, e.g., Discovery) and Techniques (how they achieve it). In our project:

- DDoS -> T1498 (Network Denial of Service)
- PortScan -> T1046 (Network Service Discovery)
- Brute Force -> T1110 (Brute Force Credentials)
- Web Attack -> T1190 (Exploit Public-Facing Application)
- Botnet -> T1071 (Application Layer Protocol C2)

Mapping every alert to MITRE ATT&CK; enables standardized forensic reporting and cross-vendor firewall rule generation.

Q3: How does your Active Host OS Containment work?

When the analyst clicks 'Block Attacker IP' in the investigation drawer (or when automated containment triggers), the backend executes a direct OS-level packet drop rule: on Windows, it invokes `netsh advfirewall firewall add rule name='SOC_BLOCK_' dir=in action=block remoteip=`. It also dynamically renders exportable Linux `iptables -A INPUT -s -j DROP` and Suricata/Snort drop signatures, demonstrating end-to-end intrusion prevention.